

8. Implementation

This chapter describes the actual implementation of the E-Commerce Multi-Vendor Platform. It highlights the major technical problems encountered during full-stack system integration, the solutions applied to resolve them, and the engineering and professional lessons gained throughout the project.

8.1 Problems Faced & Solutions

Problem 1: CORS Policy Blocking API Requests

Phase: Frontend Development (Week 7)

Description: When the React frontend client (running on `localhost:5173`) sent HTTP requests to the Django backend (running on `localhost:8000`), they were blocked by the browser with a Cross-Origin Resource Sharing (CORS) warning. Browsers block scripts from making requests to a different domain, port, or protocol unless the target server explicitly sends permission headers.

Solution: Installed the `django-cors-headers` package, registered it in Django's middleware list, and configured the client domain. Order was critical — the CORS middleware must be listed before Django's standard common middleware.

```
# backend/config/settings.py
INSTALLED_APPS = [
    'corsheaders',
    # ... other apps
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # First entry
    'django.middleware.common.CommonMiddleware',
    # ...
]

CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173"
]
```

Problem 2: JWT Access Token Not Attached to API Requests

Phase: Frontend Development (Week 7)

Description: After a successful login, the JWT access token was saved in `localStorage`. However, subsequent calls to protected endpoints (e.g., fetching order history) returned `401 Unauthorized` because the token was not attached to the request headers.

Solution: Implemented a request interceptor in the centralized Axios client file (`src/services/api.js`) to automatically copy the token from `localStorage` into the `Authorization` header before requests are sent.

```
// frontend/src/services/api.js
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Problem 3: Vendor Profile Not Auto-Created on Registration

Phase: Backend Development (Week 6)

Description: Registering as a vendor saved the User details. However, attempting to add products immediately raised a `RelatedObjectDoesNotExist: User has no vendor_profile` error because the related Vendor row was never initialized.

Solution: Added a Django `post_save` signal listener in `vendors/models.py` that automatically creates a Vendor profile row whenever a new User with `role='vendor'` is created.

```
# backend/vendors/models.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from users.models import User
from .models import Vendor

@receiver(post_save, sender=User)
def create_vendor_profile(sender, instance, created, **kwargs):
    if created and instance.role == 'vendor':
        Vendor.objects.create(
            user=instance,
            shop_name=f"{instance.username}'s Shop",
            is_approved=False
        )
```

Problem 4: OrderItem Price Stored as NULL (IntegrityError)

Phase: Backend Development (Week 6)

Description: Submitting an order failed with an `IntegrityError: NOT NULL constraint failed: orders_orderitem.price`. The `OrderItem` table requires a price value to maintain historical invoices (so a vendor updating a product's price later does not corrupt old sales records), but the order serializer was not populating it.

Solution: Updated the serializer's `create()` override to fetch the live product price and save it as a snapshot in the `OrderItem` price field at creation time.

```
# backend/orders/serializers.py
def create(self, validated_data):
    items_data = validated_data.pop('items')
    order = Order.objects.create(**validated_data)
    for item_data in items_data:
        product = item_data['product']
        price = product.price # Snapshot price
        OrderItem.objects.create(
            order=order, product=product,
            quantity=item_data['quantity'], price=price
        )
    return order
```

Problem 5: Cart Total Showing NaN in the React Frontend

Phase: Frontend Development (Week 8)

Description: The React cart page displayed the total price as `NaN` (Not a Number). The backend serializes decimal values as strings (e.g., "599.00") in JSON to prevent floating-point rounding errors. Trying to perform math operations on strings caused JavaScript to return `NaN`.

Solution: Wrapped all price value references with `parseFloat()` throughout the Cart Context and UI components before calculations are performed.

```
// frontend/src/context/CartContext.jsx
const total = cart.reduce(
  (sum, item) => sum + parseFloat(item.price) * item.qty,
  0
);
```

8.2 Lessons Learnt

Category	Key Takeaways & Engineering Insight
REST API Conventions	Using ViewSets and Serializers in DRF ensures consistent response envelopes,

	simplifies data validation, and automates typical CRUD routing.
Stateless Authentication	JWT tokens allow secure, stateless REST communication. Silent refresh flows in Axios prevent session timeouts without disrupting user activities.
Global State Management	React Context API provides lightweight, global state storage for Cart and Auth states, avoiding heavy third-party libraries (e.g., Redux) for small-to-medium systems.
Relational snapshotting	Copying the product price directly into the OrderItem table at checkout is crucial for keeping historical order records accurate if the catalog changes.
API Documentation	Auto-generating Swagger schemas using drf-spectacular makes full-stack development smoother by documenting endpoints in a central, interactive UI.

The chapter-by-chapter progression demonstrates a professional IS development methodology: from defining the problem statement in Chapter 1, to requirements capture (SRS, Chapter 3), system analysis with UML actors and use cases (Chapter 4), technical design with ERD and REST API contracts (Chapter 5), actual source code walkthroughs (Chapter 6), comprehensive testing evidence (Chapter 7), and finally conclusions with future enhancements (Chapter 8).

Architecturally, the platform implements a fully decoupled 3-tier system: a React 18 SPA (Vite) handling 20 route-bound pages and components, communicating via Axios with automatic JWT header injection and silent refresh interception, to a Django REST Framework API split into five independent apps ([users](#), [vendors](#), [products](#), [orders](#), [dashboard](#)), backed by SQLite (development) and PostgreSQL-ready for production. Key implemented features include: role-based user registration (customer/vendor/admin), vendor approval workflow, product management with image uploads (Pillow), a public Vendors/Stores browsing page, search and category filtering, client-side cart (CartContext), server-side order placement with stock deduction and price snapshots, order status lifecycle management (Pending → Shipped → Delivered), profile management for all roles, and three distinct role-specific dashboards backed by a single dynamic stats API endpoint. This implementation, supported by six code samples, demonstrates a production-ready approach to scalable e-commerce development.

